LayerZero PriceFeed

Security Assessment

June 26th, 2026 — Prepared by OtterSec

James Wang james.wang@osec.io

Table of Contents

Appendices

01 — Executive Summary

Overview

LayerZero Labs engaged OtterSec to assess the `priceFeed` program. This assessment was conducted between June 4th and June 5th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 1 finding throughout this audit engagement.

In particular, during fee estimation by EID, the hardcoded Arbitrum/Optimism fee branches are overwritten because the function continues into the eidToModelType dispatch instead of returning (OS-LPF-SUG-00).

Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/LayerZero-v2. This audit was performed against 9c741e7.

A brief description of the program is as follows:

Name	Description
priceFeed	It is LayerZero V2's on-chain fee-estimation price feed, which stores destination gas prices, calldata costs, and token price ratios, and estimates the source-chain native fee required to execute a cross-chain message. It supports default, Arbitrum-stack, and OP-stack fee models, including L1/L2 calldata overhead for rollups.

02 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

Rating criteria may be found in Appendix A.

ID	Description
OS-LPF-SUG-00	The hardcoded Arbitrum/Optimism fee branches in `_estimateFeeByEid` are overwritten because the function continues into the `eidToModelType` dispatch instead of returning.

Built-In Rollup Models Are Overwritten OS-LPF-SUG-00

Description

In `_estimateFeeByEid`, the hardcoded Arbitrum/Optimism `EID` checks compute `(fee, priceRatio)` but do not `return`, so execution always continues into the `eidToModelType[dstEid]` dispatch. If those legacy `EID`s are not explicitly configured as `ARB_STACK` or `OP_STACK`, the mapping defaults to `ModelType.DEFAULT`, overwriting the rollup-specific fee with `_estimateFeeWithDefaultModel` and under/over-pricing L1/L2 rollup costs.

>_ *packages/layerzero-v2/evm/messagelib/contracts/PriceFeed.sol* SOLIDITY

```
function _estimateFeeByEid(
    uint32 _dstEid,
    uint256 _callDataSize,
    uint256 _gas
) internal view returns (uint256 fee, uint128 priceRatio, uint128 priceRatioDenominator, uint128
    ↪  priceUSD) {
    uint32 dstEid = _dstEid % 30_000;
    if (dstEid == 110 || dstEid == 10143 || dstEid == 20143) {
        (fee, priceRatio) = _estimateFeeWithArbitrumModel(dstEid, _callDataSize, _gas);
    } else if (dstEid == 111 || dstEid == 10132 || dstEid == 20132) {
        (fee, priceRatio) = _estimateFeeWithOptimismModel(dstEid, _callDataSize, _gas);
    }

    // lookup map stuff
    ModelType _modelType = eidToModelType[dstEid];
    if (_modelType == ModelType.OP_STACK) {
        (fee, priceRatio) = _estimateFeeWithOptimismModel(dstEid, _callDataSize, _gas);
    } else if (_modelType == ModelType.ARB_STACK) {
        (fee, priceRatio) = _estimateFeeWithArbitrumModel(dstEid, _callDataSize, _gas);
    } else {
        (fee, priceRatio) = _estimateFeeWithDefaultModel(dstEid, _callDataSize, _gas);
    }

    priceRatioDenominator = PRICE_RATIO_DENOMINATOR;
    priceUSD = _nativePriceUSD;
}
```

Remediation

Ensure to return immediately from the hardcoded Arb/OP branches.

Patch

The layerzero team asserts that always continuing to `eidToModelType` is intentional.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.